< Programme Grande École — 5e année />

EpiQuest_

Une table de jeu de rôle virtuelle : personnages, cartes, monstres, dés et combats, sur deux écrans synchronisés.

Sujet de projet professionnel : alternative au part-time en entreprise.

01 Le projet

02 Le produit attendu

03 Exigences de qualité

04 Organisation et livrables

Format : groupes de 3 étudiants

Période : du 19 octobre 2026 à fin janvier 2027

Année : 2026–2027 · Epitech Paris

Auteur : Joffrey RIELA, directeur pédagogique PGE

{Epitech}

01

LE PROJET_

A. CONTEXTE

Vous développez une **table de jeu de rôle virtuelle**, dans l'esprit de D&D Beyond pour les fiches de personnage et de Roll20, Foundry ou Owlbear Rodeo pour la table de jeu. Le maître du jeu (MJ) prépare sa campagne, ses cartes et ses rencontres ; les joueurs créent leurs personnages ; puis la partie se joue **autour d'une vraie table** : le MJ pilote la partie sur son écran, et une **vue joueurs** affichée sur un second écran ou une télévision montre ce que voient les personnages.

> `< Principe />` Le produit applique les **règles de la 5e édition** publiées en licence libre (SRD). Le cœur du sujet est un **moteur de règles** piloté par les données, un **combat assisté** où le MJ garde toujours la main, et **deux vues synchronisées** : la vue joueurs n'affiche que ce que les personnages peuvent voir.

B. CADRE

- **Autonomie totale :** aucun matériel, aucune codebase et aucun apport technique ne sont fournis.
- **Stack libre**, à justifier dans la documentation technique.
- **Usage de l'IA autorisé**, encadré et déclaré (voir bloc 03-D).
- **Projet pédagogique :** le produit n'a pas vocation à être diffusé en dehors du projet.

C. RÈGLES D'OR

- **Le produit a sa propre marque.** « EpiQuest » est le nom du sujet, pas du produit. Aucune utilisation du nom ou du logo « Dungeons & Dragons », ni reprise de l'interface d'un produit existant, ni référence à Epitech. Le produit se présente comme « compatible 5e édition ».
- **Uniquement du contenu libre.** Les règles proviennent du **SRD 5e** publié par Wizards of the Coast sous licence **Creative Commons CC BY 4.0**, avec la mention d'attribution exigée. Aucun contenu issu des livres payants.
- **Visuels sous licence libre** (cartes, pions, icônes) ou créés par le groupe, crédités.

D. POINTS D'ATTENTION PARTICULIERS

Trois aspects feront l'objet d'une attention particulière (détails dans le bloc 03) :

- **Accessibilité :** des fiches, des dés, des combats et des cartes utilisables par tous (WCAG 2.2 AA).
- **Qualité du dépôt :** un dépôt propre, des bonnes pratiques, un nouveau contributeur opérationnel en 15 minutes.
- **Documentation :** utilisateur et technique.

02

LE PRODUIT ATTENDU_

A. IDENTITÉ DU PRODUIT

- Nom, logo et charte graphique **originaux**, déclinés pour le web, la table de jeu et le mobile.
- Une mini charte graphique (1 à 2 pages) est livrée.

B. PROFILS ET CAMPAGNES

- Chaque utilisateur a un compte. Un **MJ** crée une **campagne** et y ajoute les personnages de ses joueurs (par lien ou par code). Un même utilisateur peut être MJ d'une campagne et joueur dans une autre.
- Le MJ voit et contrôle tout : cartes, monstres, informations cachées. Un **joueur** crée et gère son personnage depuis le web ou l'application mobile.

C. DONNÉES DE RÈGLES

- Le contenu du SRD 5e (espèces, classes, historiques, sorts, équipements, objets magiques, monstres, états) est **importé dans la base du produit** depuis une source publique (API ouverte comme Open5e ou la D&D 5e API, ou fichiers du SRD).
- Une fois importé, le produit fonctionne **sans dépendre de l'API externe**. L'import est rejouable et versionné.
- Les règles sont **des données, pas du code** : un sort, une capacité de classe ou un monstre est décrit dans un format structuré, interprété par le moteur de règles.

D. CRÉATION ET ÉVOLUTION DES PERSONNAGES

- **Assistant pas à pas :** espèce, classe, historique, caractéristiques (achat de points, tableau standard ou jet de 4d6), compétences et maîtrises, équipement de départ, sorts.
- **Moteur de règles :** chaque choix est validé (prérequis, nombre de compétences, sorts autorisés) et les **statistiques dérivées sont recalculées** automatiquement : modificateurs, bonus de maîtrise, CA, PV, jets de sauvegarde, emplacements de sorts, poids porté.
- **Niveaux 1 à 5** pour toutes les classes du SRD, avec la **montée de niveau** : nouvelles capacités, sous-classe, augmentation de caractéristiques, nouveaux sorts.
- **Vie du personnage :** PV et PV temporaires, repos court et long, emplacements de sorts utilisés, inventaire et argent, notes.
- Fiche de personnage interactive (un clic sur une compétence lance le jet correspondant) et **export PDF**.

E. MOTEUR DE DÉS

- **Notation complète** : `2d6+3`, `4d6kh3` (garder les 3 meilleurs), avantage et désavantage, dés explosifs, relances, dés multiples dans une même expression. Les erreurs de saisie sont expliquées.
- **Jets côté serveur** avec un générateur aléatoire sûr : un joueur ne peut pas truquer ses dés.
- Jets **publics**, **privés** (visibles du MJ seul) ou **cachés** (jets du MJ). Historique complet de la session.
- Affichage des **probabilités** d'une expression (minimum, maximum, moyenne, distribution).

F. CARTES

- **Éditeur de cartes** : grille carrée ou hexagonale, image de fond, éléments de décor, **calques** (fond, décor, pions, calque du MJ invisible des joueurs).
- **Murs et portes** (ouvertes ou fermées), qui bloquent les déplacements et la vue.
- **Brouillard de guerre** : révélation manuelle par le MJ, et révélation **automatique selon la ligne de vue** des personnages, calculée à partir des murs et des portes.
- Mesure des distances selon les règles 5e, **gabarits de zones d'effet** (cône, ligne, sphère, cube) pour les sorts.

G. BESTIAIRE ET RENCONTRES

- Bestiaire issu du SRD : recherche et filtres (type, facteur de puissance, taille), fiche complète de chaque monstre.
- **Constructeur de rencontres** : sélection des monstres et calcul de la **difficulté** (facile, moyenne, difficile, mortelle) selon les règles 5e et le groupe de personnages.
- Placement de la rencontre sur une carte en un clic.

H. COMBAT ASSISTÉ

Le combat n'est **pas entièrement automatisé**. Le produit applique la **mécanique de base** décrite dans les données du SRD, et le **MJ garde toujours la main** pour ajouter ce que les règles ne prévoient pas. Les schémas de l'annexe (bloc 05, A1 à A4) illustrent ce fonctionnement.

Qui	Ce qu'il fait
Automatique (produit)	Initiative et ordre des tours. Ce qui est décrit de façon structurée dans les données : jet d'attaque contre la CA, jet de sauvegarde contre un DD, dégâts et types de dégâts (résistances, vulnérabilités, immunités), soins, zone d'effet et cibles touchées, consommation des ressources (emplacements de sorts), états standard et leur durée.
Manuel (MJ)	Un outil **« Ajouter un effet »** sur une ou plusieurs cibles : état, dégâts ou soins, modificateur temporaire, ou **effet libre** (nom, description, déclenchement, durée). Exemple : la cible d'une boule de feu « prend feu » et subit 1d4 dégâts par tour pendant 2 tours.
Toujours	Chaque résultat est affiché **avant application**, puis **modifiable ou annulable** par le MJ, et historisé dans le journal de la partie.

- Le moteur n'interprète pas le texte des sorts : chaque action est décrite avec **quelques types d'effets génériques** (attaque, sauvegarde, dégâts, soins, état, modificateur), dans un format conçu et documenté par le groupe.
- Concentration, jets contre la mort et PV temporaires sont gérés.

I. PARTIE EN LOCAL : DEUX VUES SYNCHRONISÉES

- La partie se joue autour d'une vraie table. Le MJ utilise la **vue MJ** sur son ordinateur, et ouvre une **vue joueurs** dans une seconde fenêtre, affichée sur un second écran, un projecteur ou une télévision.
- Les deux vues sont **synchronisées en direct** : déplacements, brouillard, initiative, PV, jets publics et documents révélés.
- La **vue joueurs** n'affiche que ce que les personnages voient : zones révélées, créatures visibles avec un état approximatif (indemne, blessé, à terre), jets publics. Elle ne reçoit **jamais** les informations cachées.
- Le MJ déplace les pions, ou laisse les joueurs annoncer leurs actions et les saisit pour eux.

J. GESTION DE CAMPAGNE

- Notes du MJ, PNJ, lieux et quêtes.
- **Documents à révéler** sur la vue joueurs pendant la partie (lettres, images, indices).

K. APPLICATION MOBILE

- **Périmètre :** créer son personnage (avec le même moteur de règles que le web), consulter et mettre à jour sa fiche, lancer ses dés.
- **Technologie libre :** natif (Swift, Kotlin) ou framework multiplateforme (React Native, Flutter, Kotlin Multiplatform, .NET MAUI…). Une seule plateforme cible, au choix : iOS **ou** Android. Le choix est justifié dans la documentation technique.
- **Interdit :** les sites web emballés dans une application (PWA, Ionic / Capacitor, Cordova, simple WebView affichant le site).
- Aucune publication sur les stores n'est exigée : une démo sur téléphone ou sur émulateur suffit.

L. CAMPAGNE DE DÉMONSTRATION

- Une campagne prête à jouer : **4 personnages** de classes différentes (niveaux 1 à 5), **3 cartes** (dont une avec murs et portes), une rencontre par carte.
- **Bonus** (valorisé, non obligatoire) : niveaux 6 à 20, sources de lumière et vision dans le noir, jets lancés depuis le mobile affichés sur la vue joueurs, import de cartes au format Universal VTT, ambiance sonore.

03

EXIGENCES DE QUALITÉ_

Ces exigences sont **obligatoires**. Elles comptent autant que les fonctionnalités.

A. ACCESSIBILITÉ

Le produit doit être utilisable par tous, y compris par des personnes en situation de handicap visuel, moteur, auditif ou cognitif. Le référentiel visé est le **niveau AA des WCAG 2.2**, transposé en France par le **RGAA**, sur le web et sur l'application mobile.

Exigences minimales

- **Clavier :** fiches, dés, combat et déplacement des pions utilisables sans souris, avec un ordre logique et un focus toujours visible.
- **Cartes :** une **vue alternative textuelle** accompagne la carte (pions visibles, positions, distances, zones révélées), et les événements de la partie (tour, jets, dégâts) sont annoncés aux lecteurs d'écran.
- **Lecteurs d'écran :** HTML sémantique, titres hiérarchisés, libellés sur tous les champs de la fiche.
- **Contrastes** conformes (4,5:1 pour le texte courant) et information **jamais portée par la seule couleur** (PV bas, états, alliés et ennemis).
- **Affichage :** zoom à 200 % sans perte de contenu, respect de `prefers-reduced-motion` (animations de dés désactivables).
- **Application mobile :** compatible VoiceOver (iOS) ou TalkBack (Android), tailles de texte dynamiques, zones tactiles d'au moins 44 pt / 48 dp.

Preuves attendues

- Un **audit** combinant des outils automatiques (Lighthouse, axe, WAVE…) et des tests manuels : création d'un personnage et tour de combat complets au clavier et au lecteur d'écran, sur le web **et** sur l'application mobile.
- Une **déclaration d'accessibilité** : niveau atteint, limites connues, actions prévues.

B. DÉPÔT, MÉTHODE ET BONNES PRATIQUES

> `< Test du nouvel arrivant />` Un développeur extérieur au groupe clone le dépôt. Avec le seul README, il doit pouvoir lancer le produit avec la campagne de démonstration, ainsi que les tests, **en moins de 15 minutes**.

Gestion de projet avec GitHub Projects

- Un tableau GitHub Projects lié au dépôt, **configuré et tenu à jour** du début à la fin.
- Les tâches sont des issues, avec un responsable, un label et un statut, découpées en milestones ou en itérations.
- Chaque développement passe par une **pull request** reliée à son issue et relue avant merge. La branche principale est protégée.

- L'historique doit montrer **qui a fait quoi, et quand**.

Fichiers indispensables

- `README.md` : présentation, captures, prérequis, installation, lancement, tests, liens vers la documentation.
- `CONTRIBUTING.md` : conventions et déroulé d'une contribution.
- `LICENSE`, `CHANGELOG.md`, `CODE_OF_CONDUCT.md`, et la **mention d'attribution du SRD**.
- Modèles d'issue et de pull request (`.github/`).
- `.env.example` documenté, et **aucun secret** dans le dépôt ni dans l'historique.
- `.gitignore` adapté : ni dépendances, ni builds, ni fichiers d'éditeur.

Bonnes pratiques

- Arborescence claire, sans code mort ni fichiers orphelins.
- Linter et formateur appliqués automatiquement (pre-commit ou CI).
- Convention de commits commune (par exemple Conventional Commits).
- Stratégie de branches documentée.
- Versions taguées aux étapes clés (versionnage sémantique).
- Dépendances à jour et verrouillées (lockfile).

Sécurité et robustesse

- **Le serveur fait autorité** : jets de dés et actions de combat sont calculés et validés côté serveur.
- **Aucune information cachée n'est envoyée à la vue joueurs** : pions et zones sous le brouillard, calque du MJ, jets cachés et notes privées n'y sont jamais transmis, même dans les données reçues par le navigateur.
- Permissions vérifiées côté serveur (MJ, joueur, campagne), mots de passe hachés, sessions sécurisées, protection contre les risques du **Top 10 OWASP**.
- Les deux vues restent fluides avec une carte chargée : temps de synchronisation et de calcul de la ligne de vue mesurés et documentés.

Tests

- Des tests unitaires ou fonctionnels couvrent au minimum l'**analyseur de dés** (y compris des tests statistiques de distribution), le **moteur de règles** (personnages de référence dont toutes les valeurs calculées sont vérifiées), la montée de niveau, la **ligne de vue** et le brouillard de guerre, la difficulté des rencontres, la résolution des actions de combat (automatique et effets ajoutés par le MJ, annulation) et le filtrage des informations de la vue joueurs. L'application mobile a ses propres tests (logique et interface).
- Ils se lancent en **une seule commande**.

Intégration et déploiement continus (CI/CD) obligatoire

- **Intégration continue** (GitHub Actions ou équivalent) déclenchée à chaque pull request : lint, tests du web, du serveur et du mobile, build.
- Une pull request dont le pipeline échoue **ne peut pas être mergée** (vérifications obligatoires sur la branche principale).
- **Déploiement continu :** chaque merge sur la branche principale met à jour automatiquement le produit, accessible en ligne avec la campagne de démonstration (offre gratuite d'hébergement au choix).

- **Application mobile :** chaque merge produit automatiquement un build installable ou testable (APK pour Android, build simulateur pour iOS), disponible en artefact du pipeline.
- Les secrets du pipeline sont stockés dans les secrets GitHub, jamais dans le code.
- Le pipeline est décrit dans la documentation technique, et son badge d'état figure dans le README.

Accueil d'un nouveau contributeur

- **Installation en une commande** (Docker Compose, devcontainer ou script), avec import du SRD et chargement de la campagne de démonstration. La procédure pour lancer l'application mobile sur émulateur est documentée.
- Un **guide de prise en main** : où se trouve quoi, comment le moteur de règles interprète une capacité, comment lancer un seul test.
- Des issues `good first issue` et une roadmap visible dans GitHub Projects.
- Des messages d'erreur et des logs compréhensibles en développement.

C. DOCUMENTATION

La documentation fait partie du produit et s'écrit au fil du projet. Elle est versionnée dans le dépôt (`/docs`, en Markdown) ou publiée depuis celui-ci (GitHub Pages, Docusaurus, MkDocs…). Langue au choix, mais la même partout.

Documentation utilisateur

Pour des personnes non techniques : elle permet d'utiliser le produit sans aide.

Public	Contenu attendu
Maître du jeu	Créer une campagne et ajouter les personnages, dessiner une carte (murs, portes), préparer une rencontre, afficher la vue joueurs, mener un combat et ajouter des effets, révéler des documents.
Joueur	Créer et faire évoluer son personnage, lancer des dés, rejoindre une campagne, utiliser l'application mobile.
Pour tous	Guide pas à pas illustré, aide-mémoire de la notation des dés, FAQ, fonctionnalités d'accessibilité disponibles.

Documentation technique

Pour un développeur qui reprendrait le projet : elle permet de comprendre, modifier et déployer le produit.

Rubrique	Contenu attendu
Architecture	Schéma des composants (web, serveur, vues MJ et joueurs, moteur de règles, mobile), flux de données, choix de stack justifiés.
Décisions	Journal des décisions d'architecture (ADR) : contexte, options, choix retenu, conséquences.
Modèle de données	Schéma des entités (campagne, utilisateur, personnage, classe, espèce, sort, objet, monstre, carte, calque, pion, rencontre, combat, jet) et dictionnaire des champs.
Données de règles	Import du SRD, format de description des règles, versionnement.
Moteur de règles	Calcul des statistiques dérivées, validation des choix, montée de niveau, format des effets génériques, résolution et annulation des actions de combat.
Dés	Grammaire de la notation, générateur aléatoire, calcul des probabilités.
Cartes et vision	Modèle des cartes et calques, algorithme de ligne de vue, brouillard de guerre, filtrage des informations par joueur.
Synchronisation des vues	Mécanisme de synchronisation entre la vue MJ et la vue joueurs, données transmises à chacune.
Application mobile	Architecture de l'app, échanges avec l'API, lancement et build.
Exploitation	Déploiement, pipeline CI/CD, résultats des mesures de performance et de l'audit d'accessibilité.

D. USAGE DE L'IA

L'usage d'outils d'IA (assistants de code, génération de contenu…) est **autorisé**. Il est encadré par les règles suivantes, conformes à la charte IA du groupe IONIS.

Transparence

Une section du README déclare les outils utilisés et pour quoi : code, tests, documentation, visuels.

Responsabilité

Le groupe est responsable de tout ce qu'il livre. Chaque membre doit pouvoir **expliquer n'importe quelle partie du code** en soutenance.

Même exigence

Le code et les contenus générés respectent les mêmes règles que le reste : tests, revue en pull request, propreté du dépôt, sécurité.

Licences

Les images, cartes et textes générés ou récupérés respectent les licences et les règles d'or. Aucun contenu des livres payants n'est reproduit.

04

ORGANISATION ET LIVRABLES_

A. CALENDRIER

19 octobre 2026

LANCEMENT

Présentation du sujet et constitution des groupes.

Courant décembre

FOLLOW-UP

Facultatif. Point d'avancement et démo de l'existant, pour recueillir des retours.

Fin janvier 2027

SOUTENANCE

Création d'un personnage sur mobile, puis combat sur deux écrans avec le jury comme joueurs, parcours accessible, questions sur le code.

Courant février

RATTRAPAGE

Réservé aux projets non aboutis fin janvier.

B. LIVRABLES

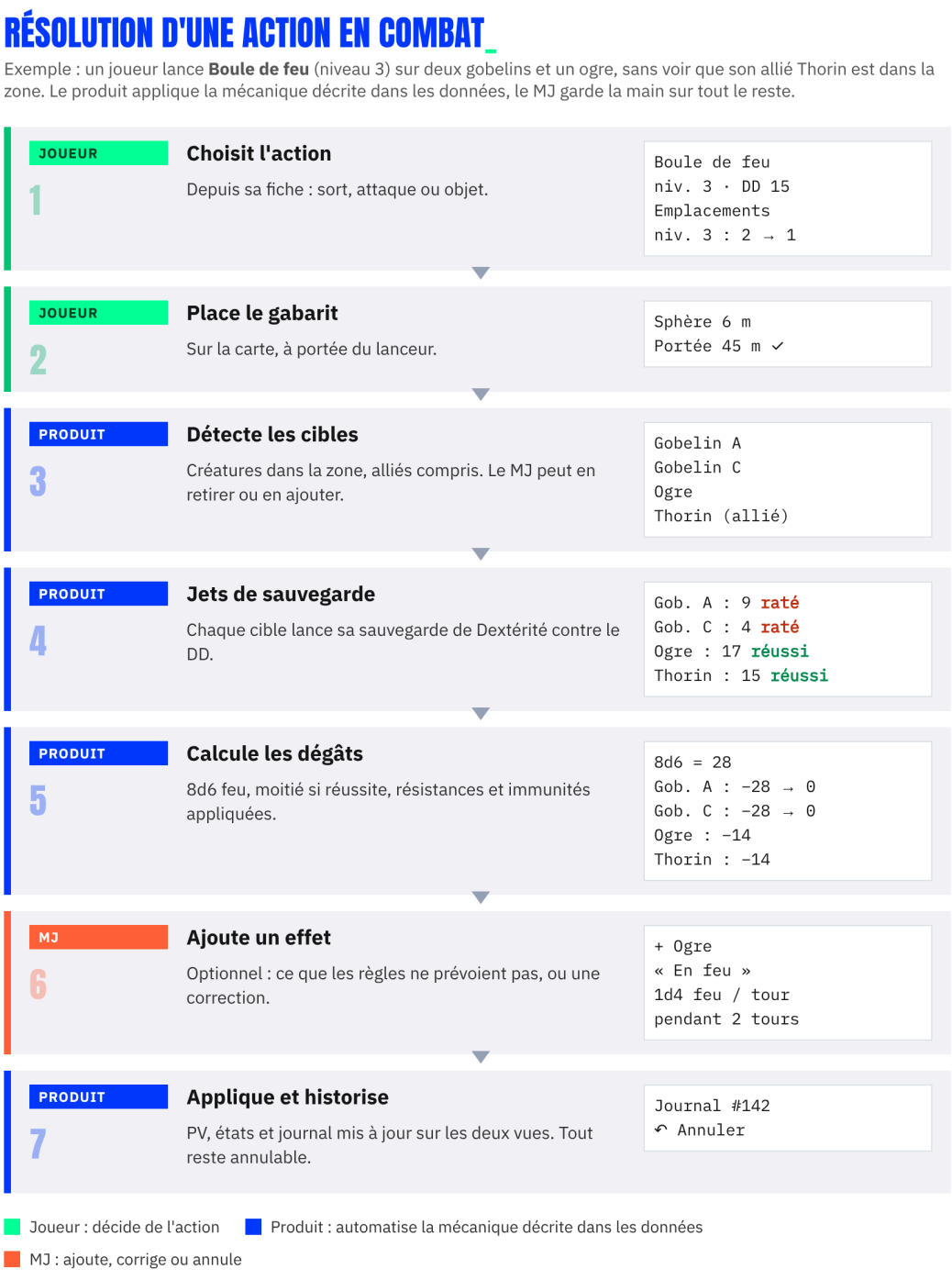
Livrable	Contenu
Produit web	Personnages, dés, cartes, bestiaire, rencontres, combat assisté, vue MJ et vue joueurs synchronisées, gestion de campagne.
Application mobile	Création de personnage, fiche et dés, sur iOS ou Android, sans site emballé.
Données de règles	Import du SRD 5e rejouable, avec la mention d'attribution.
Campagne de démo	Quatre personnages, trois cartes et leurs rencontres.
Environnement en ligne	Produit déployé automatiquement avec la campagne de démonstration, URL fournie dans le README.
Dépôt GitHub	Propre, avec les fichiers indispensables, les tests, le pipeline CI/CD et le tableau GitHub Projects.
Identité du produit	Mini charte graphique (1 à 2 pages).
Documentation	Utilisateur et technique.
Accessibilité	Audit (web et mobile) et déclaration d'accessibilité.
Usage de l'IA	Déclaration dans le README.
Soutenance	Présentation et démo.

05

ANNEXE : COMBAT ASSISTÉ_

Ces schémas illustrent le fonctionnement attendu sur un exemple. **Ils sont indicatifs** : la disposition, le style, les libellés et le format des données sont libres.

A1 · Résolution d'une action en combat



MAQUETTE INDICATIVE : VUE DU MJ PENDANT LE COMBAT_

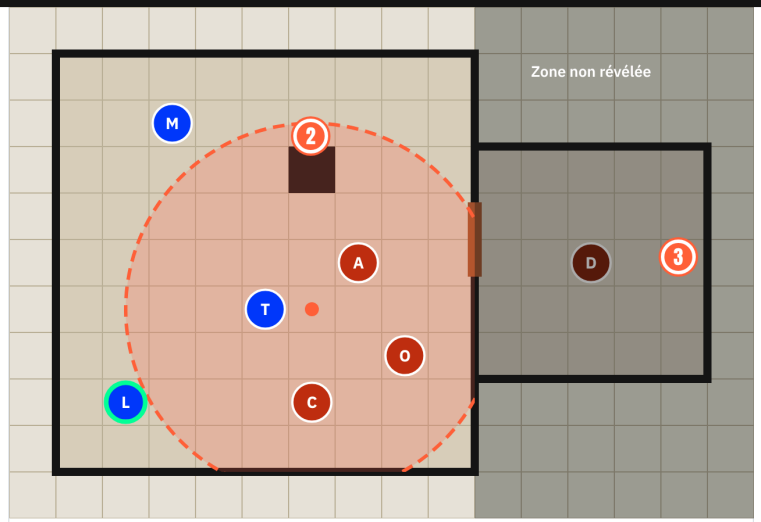
À titre d'exemple : la disposition, le style et les libellés sont libres. Ce qui compte, ce sont les informations et les actions disponibles.

[VOTRE PRODUIT] — Campagne « La Crypte oubliée » · Combat · Round 2 — VUE MJ

INITIATIVE (1)

18	**Lyra (magicienne)**	22/22
15	~~Gobelin A~~	0/7
14	Thorin (guerrier)	20/34
12	Ogre (en feu)	45/59
11	Mira (clerc)	27/27
9	~~Gobelin C~~	0/7
7	Gobelin D (caché)	7/7

[Tour suivant ▸] [+ Créature]



Légende : ● personnages (L = Lyra, T = Thorin, M = Mira) · ● créatures (A, C = gobelins, O = ogre, D = gobelin caché) · cercle pointillé : gabarit de la Boule de feu (sphère de 6 m), arrêté par les murs · zone grisée : non révélée aux joueurs

RÉSULTAT : BOULE DE FEU (LYRA) · DD 15 · 8D6 = 28 (4)

Cible	Sauv. DEX	Dégâts	PV
Gobelin A	9 · raté	28	7 → 0
Ogre	17 · réussi	14	59 → 45
Gobelin C	4 · raté	28	7 → 0
Thorin	15 · réussi	14	34 → 20

[✓ Appliquer] [✎ Modifier] [+ Ajouter un effet] [↶ Annuler]

Ajouter un effet (5)

Type : [État] [Dégâts] [Soins] [Modificateur] [Effet libre]
Cibles : Ogre ×
Nom : En feu
Effet : 1d4 dégâts de feu
Quand : Début de chaque tour de la cible
Durée : 2 tours
[Ajouter]

JOURNAL (6)

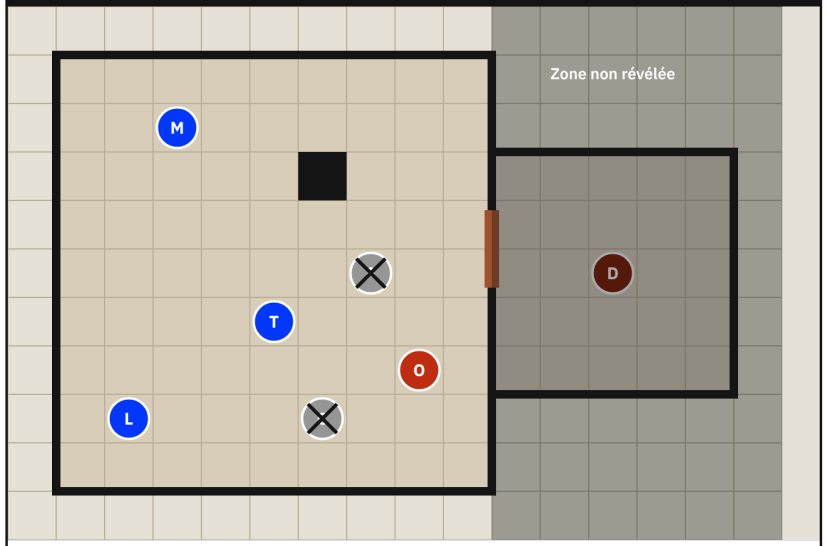
```
#141 Lyra lance Boule de feu (niv. 3)
#142 8d6 → 28 · 4 cibles · appliqué
#143 MJ : « En feu » → Ogre
```

1. **Initiative :** ordre des tours calculé automatiquement, tour courant mis en évidence, PV et états visibles. Le MJ voit aussi les créatures cachées.
2. **Gabarit :** zone d'effet placée par le joueur ; les créatures touchées sont détectées, alliés compris.
3. **Calque du MJ :** le gobelin D derrière la porte est visible pour le MJ uniquement.
4. **Résultat automatique :** sauvegardes, dégâts et PV calculés à partir des données du sort, avant application.
5. **Ajouter un effet :** le MJ complète ce que les règles ne prévoient pas (état, dégâts, soins, modificateur ou effet libre, avec cibles et durée).
6. **Journal :** chaque action est historisée et peut être annulée.

DEUX VUES SYNCHRONISÉES, UNE SEULE PARTIE_

Le MJ joue sur son écran. La vue joueurs est affichée sur un second écran ou une télévision, et se met à jour en direct. Elle ne reçoit jamais les informations cachées.

VUE MJ — écran du MJ — TOUT EST VISIBLE



CRÉATURES

Ogre (en feu)	45/59 PV
Gobelin D (caché)	7/7 PV

NOTES DU MJ

Le coffre de la salle est piégé (DD 13).

⇄ synchro en direct

VUE JOUEURS — second écran / télévision — CE QUE VOIENT LES PERSOS



GROUPE

Thorin	20/34 PV
Ogre	Blessé, en feu
Gobelins A et C	Hors de combat

DERNIERS JETS

Lyra · Boule de feu · 28 dégâts

Information	Vue MJ	Vue joueurs
Zones de la carte	Toute la carte, zones non révélées grisées	Uniquement les zones révélées par le MJ ou visibles par les personnages (ligne de vue)
Créatures	Toutes, y compris cachées, avec PV exacts	Seulement celles visibles, avec un état approximatif (indemne, blessé, à terre)
Jets de dés	Tous, y compris les jets cachés du MJ	Jets publics uniquement
Calque du MJ, notes, pièges	Visibles	Jamais envoyés à la vue joueurs
Documents	Bibliothèque complète	Seulement les documents que le MJ choisit d'afficher

CE QUE LE MOTEUR DE RÈGLES DOIT COMPRENDRE_

Le moteur n'interprète pas le texte des sorts. Chaque sort, attaque ou capacité est décrit avec **quelques types d'effets génériques**, qui couvrent la grande majorité des cas. Le reste est ajouté par le MJ.

STRUCTURE D'UNE ACTION

Action
sort, attaque d'arme, capacité, objet · coût (emplacement de sort, utilisation) · portée

Ciblage
`cible unique` `plusieurs cibles` `zone : sphère, cône, ligne, cube` `soi-même`

Résolution
`jet d'attaque contre la CA` `jet de sauvegarde contre un DD` `automatique`

Effets si touché / si raté
`dégâts (dés + type)` `soins` `état + durée` `modificateur temporaire` `moitié des dégâts`

Durée et concentration
instantané, nombre de tours, jusqu'à la fin d'un repos · concentration requise ou non

Effets ajoutés par le MJ
mêmes types d'effets, plus un **effet libre** (nom, description, déclenchement, durée), appliqués à la main sur les cibles de son choix

EXEMPLE DE DESCRIPTION STRUCTURÉE

```
// Boule de feu : description indicative, format libre
{
  "nom": "Boule de feu",
  "type": "sort", "niveau": 3,
  "cout": { "emplacement": 3 },
  "portee_m": 45,
  "ciblage": { "zone": "sphere", "rayon_m": 6 },
  "resolution": {
    "sauvegarde": "DEX",
    "dd": "lanceur.dd_sorts"
  },
  "si_rate":   [{ "degats": "8d6", "type": "feu" }],
  "si_reussi": [{ "degats": "moitie" }],
  "niveau_superieur": { "par_niveau": "+1d6" },
  "description": "Texte du SRD, affiché au joueur"
}
```

Le format exact est à concevoir par le groupe et à documenter. Il doit permettre de décrire les sorts et attaques du SRD sans écrire de code spécifique pour chacun.